

Project - Spam Detection Application

Deep learning AIMS F24

Sin Tung Jane WU

Introduction

The idea of this project is to use deep learning to build a model that enables users to detect whether the email is spam or not. And by building an application using Streamlit, this model can be put into practice.

Remarks:

For this project, I used two different platforms: Google Colab for preprocessing and VS Code for building and running the Streamlit application.

I chose Google Colab for preprocessing because I needed to use the NLTK library to analyze word frequency. Unfortunately, on my Mac, I wasn't able to install NLTK properly in VS Code due to outdated certificate issues (but it worked on a Windows laptop). To keep things simple and efficient, I stuck with Colab, which has NLTK pre-installed and ready to use.

As for building the Streamlit app, I switched over to VS Code because Google Colab doesn't support running local servers, which Streamlit needs to function. While Colab can generate a temporary URL using tools like pyngrok, it didn't consistently work for me. So, at the end of my preprocessing in Colab, I added a few lines of code to automatically download the trained model (in .h5 format) and the tokenizer (in .pkl). I then transferred those files — along with the dataset, the NLTK punkt_tab_english.pickle, and my preprocessing script — into VS Code. This setup allowed me to run the Streamlit app locally and test it properly.

Dataset acquisition

The dataset is acquired from Kaggle, which has around 5000 entries. These emails are collected through an academic paper: *V. Metsis, I. Androutsopoulos and G. Paliouras, "Spam Filtering with Naive Bayes - Which Naive Bayes?". Proceedings of the 3rd Conference on Email and Anti-Spam (CEAS 2006), Mountain View, CA, USA, 2006.*

Unlike many other email datasets that have significant class imbalance (e.g., 10% spam and 90% ham), this dataset provides a more balanced distribution — closer to 70% ham and 30% spam. This helps reduce the risk of bias or overfitting during model training, enabling the classifier to generalize better across both classes.

The second reason is the inclusion of the subject in the email. A unique feature of this dataset is that it contains both the subject line and the body of the email. Many other datasets only include the body content. Including the subject line may help improve model performance, as subjects often carry strong spam indicators such as “Free,” “Congratulations,” or “Act Now.”

The third reason is because of the sufficient dataset size for deep learning. With over 5,000 entries, the dataset is large enough to effectively train a deep learning model such as an LSTM, while still being small enough to allow for fast experimentation and prototyping. It provides a good trade-off between data richness and training efficiency.

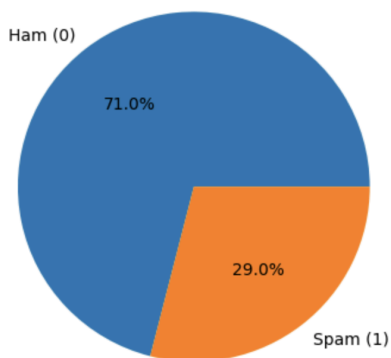
EDA

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 5171 entries, 0 to 5170  
Data columns (total 4 columns):  
#   Column      Non-Null Count  Dtype    
--  --        
0   Unnamed: 0   5171 non-null   int64    
1   label        5171 non-null   object   
2   text         5171 non-null   object   
3   label_num    5171 non-null   int64    
dtypes: int64(2), object(2)  
memory usage: 161.7+ KB
```

This dataset has 5171 entries in total, with 4 columns: unnamed, label, text and label_num. The unnamed column can be dropped as it indicates nothing. And for labels, it indicates whether the email is spam or ham. And label_num is the binary encoding of the label, which is 0 (ham) and 1 (spam) and this is our target. We will keep only the label_num column and drop the label column as it is the same but the label_num has been encoded. The text is the content of the email which is an object.

Distribution of spam and ham email



The distribution of spam and ham email in this dataset is 71% ham and 29% spam email. This is the least imbalance dataset that I can find. Therefore, in the data splitting part, we know the dataset is not thaaaaat balance (30% vs 70%), therefore I decided to split the data to 70% training, 15% validation and 15% test set.

	num_words	num_sentence
count	3672.000000	3672.000000
mean	226.239107	11.440904
std	332.179661	24.892442
min	3.000000	1.000000
25%	51.000000	3.000000
50%	124.000000	6.000000
75%	294.000000	13.000000
max	8863.000000	1204.000000

Ham emails

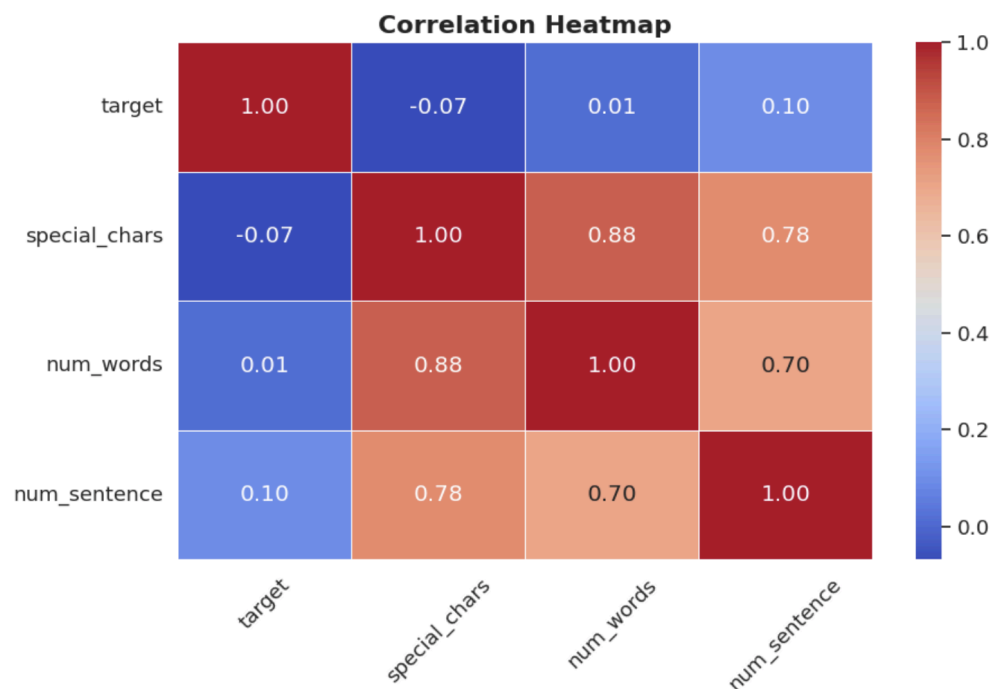
	num_words	num_sentence
count	1499.000000	1499.000000
mean	236.387592	17.237492
std	346.821249	32.796043
min	2.000000	1.000000
25%	55.000000	4.000000
50%	116.000000	9.000000
75%	244.000000	18.500000
max	3963.000000	693.000000

Spam emails

I used the `nltk.word_tokenize()` and `nltk.sent_tokenize()` to analyze the number of sentences and word count in a text and so I can separate it into 0 and 1 to see the characteristics of it. As we can see, the mean number of words on spam email is more than ham email, this means that generally spam emails have more words than ham emails. Moreover, spam emails have more number of sentences.

Type	Avg special character
Spam email	50.15
Ham email	70.08

And by using the string.punctuation function, I am able to see how many special characters are in spam and ham emails. We can see that in ham email, generally there are more special characters, as most of the email companies have the spam email filter which if there are too many special characters in an email, it will trigger the filter. Therefore, spam email has less special character.



From the correlation heatmap, we can see that the features do not have high correlation with the target. However we observe that emails with more sentences tend to have more words and more special characters.

Data cleaning

```
print(df.isnull().sum())
print(df.duplicated().sum())
print(df.info())
```

There is no missing value as we can see is non-null. As well as there is no duplicate.

```
text      0
target    0
num_words 0
num_sentence 0
special_chars 0
dtype: int64
610
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5171 entries, 0 to 5170
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    text        5171 non-null   object
1    target      5171 non-null   int64
2    num_words   5171 non-null   int64
3    num_sentence 5171 non-null   int64
4    special_chars 5171 non-null   int64
dtypes: int64(4), object(1)
memory usage: 202.1+ KB
None
```

As in the dataset, the subject of the email is included in the text, therefore, I will need to extract the subject from the text by using the for loop. Once it is separated, I will create a new column for the subject using the .append(). After extracting the subject from text, I deleted the subject in the text to avoid duplicating the subject.

Changing uppercase to lowercase

By using the `.str.lower()`, I am able to turn all the uppercase letters in the text to lowercase. The reason for doing this is to prevent duplication of word representations. For example, if I did not do this step, the model may think “Word” and ‘word’ are two different words, even though they have the same meaning. Thus, lowercasing the words can ensure consistency.

Deleting punctuation marks

The reason for deleting punctuation marks is to remove noise and irrelevant characters. As punctuation marks generally don't carry significant semantic meaning in text classification tasks. Removing them helps the model focus on the actual words.

Deleting Stopwords

By using NLTK, I just have to download the NLTK English stopwords which helps me to identify stopwords in the “text” and remove common words with little prediction power. For example the stopwords like “a”, “is” appear frequently in a text but these stopwords are not helping to distinguish if the email is spam or ham email. So removing it can help to reduce the noise.

Deleting numbers

By using the `.str.replace(r'\d+', '', regex=True)`, and `\d` represent digits (0-9). And `''` helps to empty the string, which is deleting the numbers in the text.

Top 10 most frequent words

By importing Counter, which is a useful tool for counting the frequency of items.

ect: 13893	com: 990
hou: 7281	http: 983
enron: 6279	company: 723
subject: 2728	e: 606
please: 2697	www: 587
com: 2693	information: 513
deal: 2655	font: 511
gas: 2629	td: 504
cc: 2357	statements: 476
pm: 2325	please: 473

Ham email

Spam email

We can see that for ham email, the top 10 most frequent wordings are mostly the short form of some words, for example, pm = private message. This makes sense as ham email involves more human interaction. For spam email, the top 10 words are mostly words that are in url like www, com, http. As they want readers to click into their link to scam them, that is why it is so often that it includes a link in spam emails.

Preprocessing

Train test split

For preproces, I first split the data into X and y. In X, it only includes the 'text' and y is our target (which is 0/1). I will explain the reason for only including the "text" column in the training in the model testing section.

For splitting into test sets and training sets, I decided to split the data to 70% training, 15% validation and 15% test set. The reason for doing this is because the dataset is 70% of ham email and 30% of spam email. This also helps for preventing overfitting, as I ensure each set is large enough to represent both spam and ham examples and gives more reliable feedback to avoid overfitting to the training set.

Text Vectorization

I used the TextVectorization layer from Keras which automatically handles both tokenization and vectorization of text data. Max_token = 10000 which means it controls the maximum number of unique words or sub-words as 10000 that will be used to represent the text data.

For the output_sequence_length I set it as 478. By using the .split(), splitting it into words and then counting the number of words using .len(). Then, calculate the 95th percentile which is 478. This means that 95% of the spam emails have a length less than or equal to 478. This is super important for feeding data into neural networks that require consistent input shapes as well as in building the Streamlit application.

Embedding

After text vectorization, I did embedding which helps to represent the text into numerical vectors. The input dimension=vocab_size, which is the number of unique words (tokens) in the vocabulary that the TextVectorization layer has learned in the previous step.

For output dimension = 50 as it gives a good balance between representational power and computational efficiency. And I did not use a higher output dimension because higher dimensions are usually used for larger dataset.

Model training

I used LSTM as the model architecture for this spam email detector. The reason for using it is because emails are inherently sequential, meaning the order of words matters for understanding the message's meaning and LSTMs are designed to process sequences. Moreover, the meaning of a word in the email also depends on the words surrounded, and LSTM can have the memory of the previous words which helps them understand the context and relationships between words.

I used 32 as the number of LSTM units in the layer. And using kernel_regularizer=l2(0.01) helps prevent overfitting . By keeping weights small, L2 regularization helps to reduce the complexity of the model. It prevents the model from relying too heavily on individual features or learning overly specific patterns in the training data. And I set the drop out as 0.7 in order to force the model to learn more robust patterns and not memorize the training data (prevent overfitting), I will explain later why I put the dropout rate so high.

For the final prediction layer, I used sigmoid as activation as the target is a binary classification and the sigmoid function outputs a value between 0 and 1.

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	(None, 478, 50)	500,000
lstm (LSTM)	(None, 32)	10,624
dropout (Dropout)	(None, 32)	0
dense (Dense)	(None, 1)	33

Total params: 510,657 (1.95 MB)
Trainable params: 510,657 (1.95 MB)
Non-trainable params: 0 (0.00 B)

This is the model summary which:

Layer 1: Embedding. The param is 500,000 as $\text{input_dim} = 10,000$ and size of each word embedding = 50, so the number of parameters = 10000×50 . 478 is the `max_length` that I defined earlier.

Layer 2: LSTM. It reads the entire sequence and captures patterns and context using 32 memory neurons. The param is 10,624 as $(50 + 32 + 1) \times 32$.

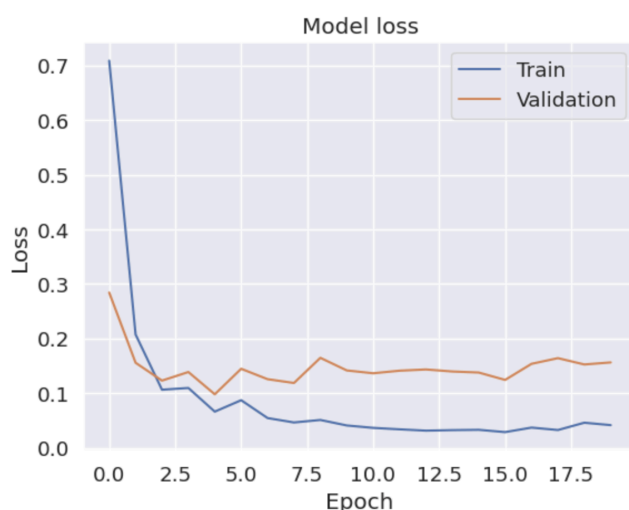
Layer 3: Dropout. Randomly "drops" 70% of the LSTM outputs during training to prevent overfitting.

Layer 4: Dense. Outputs a single value per email: 1 = spam, 0 = ham. The param is 33 parameters as 32 weights from LSTM + 1 bias.

Then, I used `.BinaryCrossentropy()` as I am solving a binary classification problem: spam (1) vs not spam (0). `BinaryCrossentropy` compares the model's output (a probability between 0 and 1 from the sigmoid) to the true label (0 or 1).

The learning rate is 0.001 which is not too fast to overshoot, not too slow to get stuck. And I also added Early Stopping which stops training when it starts to increase, preventing the model from overfitting to the training data.

And start training the training set, I set batch size = 32 (even though it is a Smaller batch size which leads to slower training but improves the model's performance) and epoch = 20 (meaning the training dataset is used to train the model 20 times).



We can see that the model is still a little bit not stable but there is less overfitting as we can see the gap between train and validation is not large.

For the F1 score on the test set, it is 0.92, which is not that bad but it can be improved.

Model testing

This section will explain how I come up with the final model which is:

```
model = tf.keras.models.Sequential([
    tf.keras.layers.InputLayer(input_shape=(max_length,)),
    embedding_layer,
    tf.keras.layers.LSTM(32, return_sequences=False,
kernel_regularizer=tf.keras.regularizers.l2(0.01)),
    tf.keras.layers.Dropout(0.7),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

history = model.fit(X_train_vect, y_train,
                    batch_size = 32,
                    epochs = 20,
                    validation_data=(X_val_vect, y_val))
```



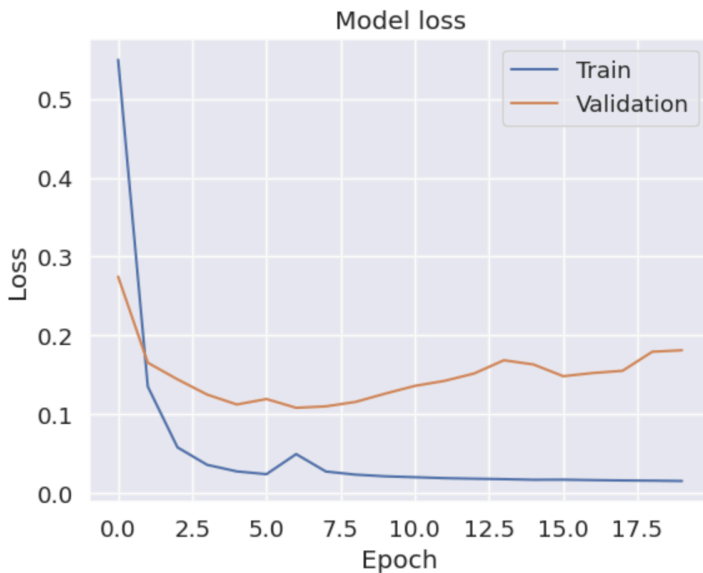
Trial 1: Dropout from 0.2 → 0.5

We can see there is a big gap between train and validation which indicates that there is an overfitting, especially after epoch 5. Therefore, I changed the dropout from 0.2 to 0.5, which may help the model to learn strictly.



Trial 2: Changing LSTM layers

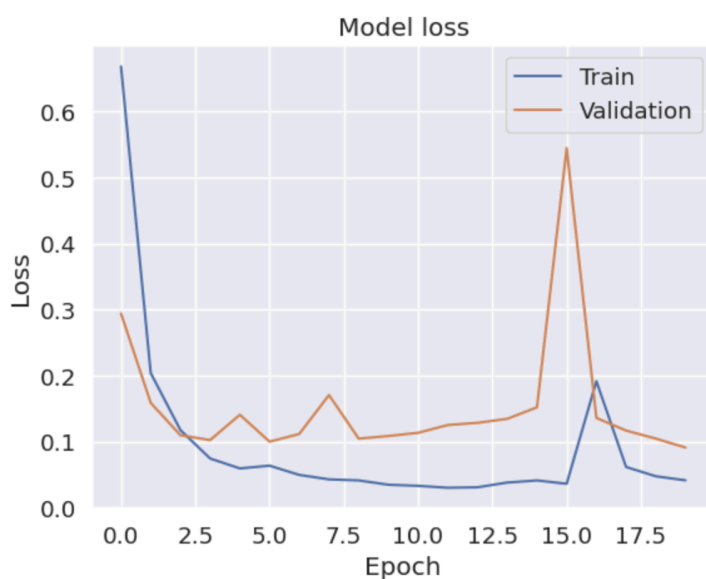
However, after changing the dropout from 0.2 to 0.5, still overfitting, Therefore, I changed the LSTM layers from 64 to 32. The reason why I think this may deal with overfitting as the model with 64 LSTM units has more capacity to "memorize" the training data, including noise or irrelevant details. By reducing the number of LSTM units to 32, the model becomes simpler and less relying on memorization. This helps the network to learn more general patterns.



Trial 3: Changing batch size and adding early stopping

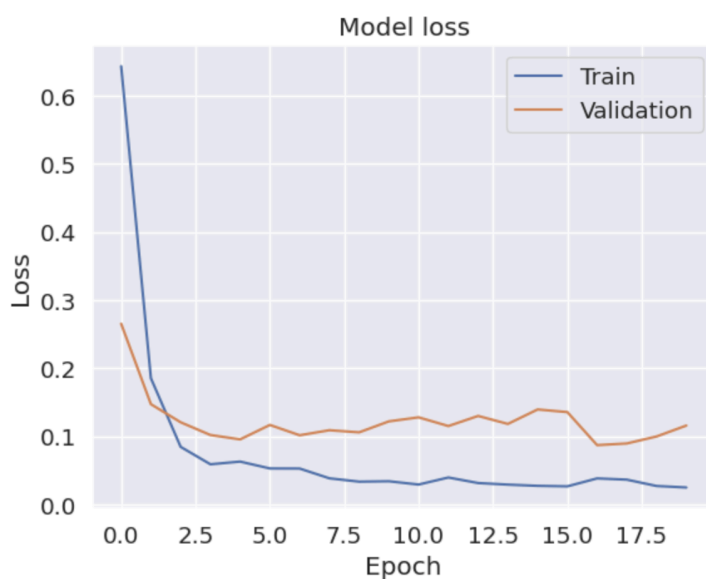
Reducing the batch size from 50 to 32 and epochs keep at 20. Smaller batch size helps the model to escape sharp local minima and generalize better — leading to a smoother validation loss curve.

For adding an early stopping, it stops training when it starts to increase, preventing the model from overfitting to the training data.



Trial 4: Only include “text” as the feature

Originally, I standardised the numerical features which are the num_words and num_sentence and included it in the training. I decided to include only X_train_vec (which is only the “text”) . However, while this helped with overfitting, the validation loss became less stable, fluctuating more across epochs.

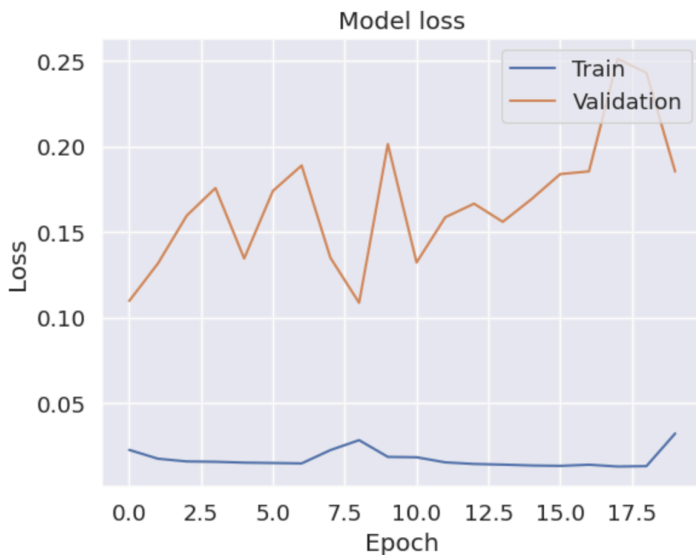


Trial 5: Dropout from 0.5 to 0.7

Increasing the dropout rate means that more neurons are being dropped out during training. This introduces more noise (which then the validation set will be harder to memorise the training set and so less overfitting) and reduces co-adaptation further, potentially leading to a more stable model. And the result is yes, it is more stable now.

Trial 6: Batch size from 32 to 16

I tried to change the batch size from 32 to 16, but when the batch size is too small (like 16), means more weight updates per epoch, which can lead to faster overfitting — the model over-adapts to the training data too quickly. Therefore, we will stay with batch size = 32.



Building application

Preparation for Streamlit

I used VS Code to build the application as it can connect to a local tunnel to run the application. In order to do this, I added `files.download()`, so when I run the code on Google Collab, it will automatically download the model as `.h5` and `tokenizer.pkl` in my pc.

In VS Code, I created a file with the `.h5` and `tokenizer.pkl` as well as the preprocess in `.ipynb`, the dataset, punkt tab for English and created a new one for building the application using Streamlit.

Then I went to <https://docs.streamlit.io/get-started/tutorials/create-an-app> to have more understanding about how to use Streamlit. I first import all the necessary ones which are streamlit, streamviz (for visualisation), tensorflow (for the deep learning model), numpy and pandas.

First, I have to load the model that I have done on Google Collab by using `tf.keras.model.load_model`. As well as the `tokenizer.pkl` that I downloaded on Google Collab. This is used for NLP to convert text into numerical data so the model can understand. And define the `max_length` which is 478, this will be useful later for preprocessing the input.

Design user interface

Then, I am now designing the user interface, the page should be separated into left and right. The left side shows the result of the prediction whether it is a spam or not spam and also with the spam score (lower value = is not spam). And on the right hand side, it allows users to input their text and press the run button.

Title: Using `st.set_page_config` to set up the title of my page with the layout = “centered”. Then, use `st.markdown()` to display the title of the application in the user interface. With `h1` style (meaning this title is a heading), center and in `#333` a dark charcoal colour.

Layout: Using the `col1, col2 = st.columns([1,2])`, to divide the page into left and right with a ratio of 1:2, this can help to give a better visual. Then, within `col2` (the right hand side of the page) use `st.text_area()` to allow user to input their texts and `st.button()` to create a button for users to run it.

Make prediction

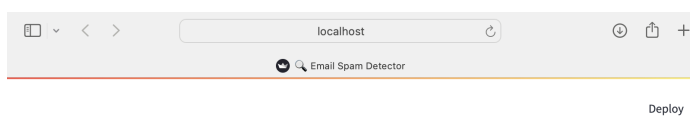
First, after the user input the text, by using the `if run and email_input.strip() != ''`, the user has to press the run button and if run button has been pressed and there is content in the input area, then the model will start to run.

Then, I will have to preprocess the input text in the same way for the preprocess of training the model which are: lowercase the words, remove number, remove punctuation and remove stopwords. This step is to ensure the consistency of the input text is preprocessed the same way as when I was training the model.

After that, `prediction = model.predict(padded)[0][0]` which the preprocessed input will pass through to the model and prediction is the probability score. And confidence is the percentage and rounded to 2 decimal places, which is easier for users to understand.

It will be determined as a spam email if the prediction is equal or higher than 0.5. And also created the label for spam email and not spam email. And within `col1` (which is the left hand side), it will display the result by using the `st.markdown` as a bigger font size for `col1` and then `st.write` to display the result.

The final results are as below:



Email Spam Detector

Prediction

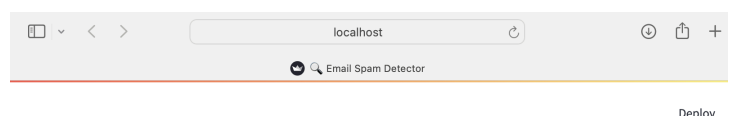
Result:  SPAM

Spam Score: 99.87

Write your email content below 

discount drugs . . . save 80 % every order !
we are the number one online retailer for dozens of medications
. our customers save 80
cents out of every dollar , every time , compared to the industry
price . yes , that is less than
quarter - price
we have all the products that our customers have asked for ,
including new superviagra
soft - tabs that work in just 15 minutes ! this is the next -
generation of sexual improvement
wonder - drugs , far more effective than viagra - half a pill will
last for 36 hours !
get all the information on superviagra here : <http://cordoned .>

Run



Email Spam Detector

Prediction

Result:  NOT SPAM

Spam Score: 1.74

Write your email content below 

hi guys ,
please take a look at the following meters which are showing up
with
variances on my ua 4 report . once the variance has been cleared ,
please send
me an e - mail . please try to clear these within the next couple of
days .
thanks ,

Run